Homework 11

Solutions will be available after Monday, November 9, 2026.

Question 1: Implementing K-means

K-means is a relatively simple algorithm that we can write ourselves. For Questions 1 and 2, do not use an existing function that performs K-means directly.

Generate 10 observations with two independent standard normal coordinates. Set a random seed and report it. Randomly assign the observations to $K = 2$ clusters, making sure neither cluster is empty. Let $C(i)$ denote the cluster assigned to observation x_i, and let m_k denote the mean of cluster k.

```
set.seed(2024)   # seed used for the data and for all random assignments
n <- 10
p <- 2
K <- 2

X <- matrix(rnorm(n * p), nrow = n, ncol = p)
colnames(X) <- c("x1", "x2")
```

The random seed is 2024. The data are 10 observations with two independent standard normal coordinates.

```
# Random assignment where no cluster is empty
random_assignment <- function(n, K) {
  repeat {
    a <- sample(1:K, n, replace = TRUE)
    if (length(unique(a)) == K) return(a)
  }
}

# Cluster means. An empty cluster keeps its previous mean.
cluster_means <- function(X, assign, K, prev_means = NULL) {
  m <- matrix(NA_real_, nrow = K, ncol = ncol(X))
  for (k in 1:K) {
    if (any(assign == k)) {
      m[k, ] <- colMeans(X[assign == k, , drop = FALSE])
    } else {
      m[k, ] <- prev_means[k, ]
    }
  }
  m
}

# Assign each observation to the closest mean (Euclidean distance)
closest_mean <- function(X, means) {
  K <- nrow(means)
  d <- matrix(NA_real_, nrow = nrow(X), ncol = K)
  for (k in 1:K) {
    diffs <- sweep(X, 2, means[k, ], "-")
    d[, k] <- sqrt(rowSums(diffs^2))
  }
```

```
  apply(d, 1, which.min)
}
```

a. Perform one iteration: calculate the cluster means, then assign each observation to its closest mean using Euclidean distance. Report the means and updated assignments, and plot the observations colored by their updated assignments.

```
init <- random_assignment(n, K)
init
```

```
 [1] 2 2 1 1 1 2 2 2 2 1
```

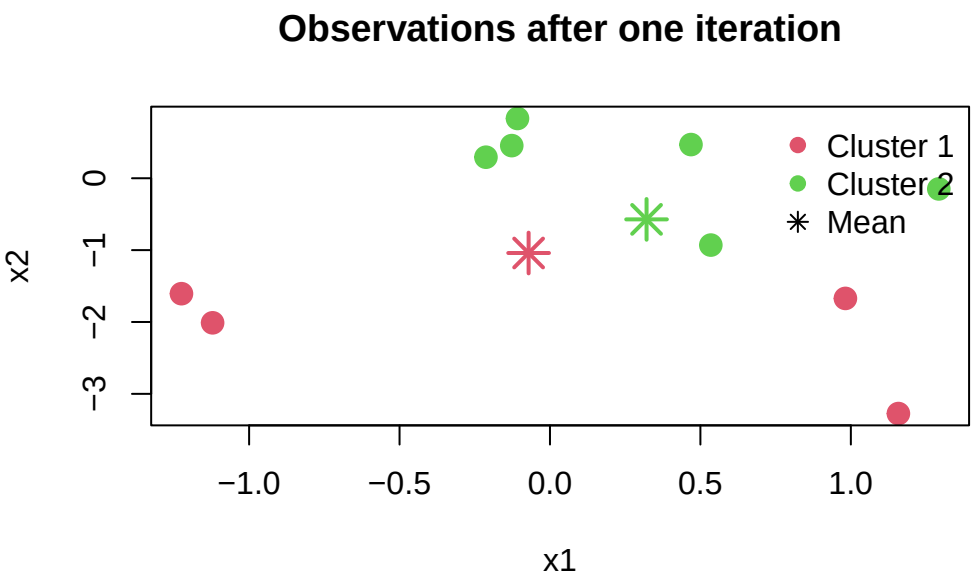
```
means_a <- cluster_means(X, init, K)
means_a
```

```
           [,1]       [,2]
[1,] -0.07102559 -1.0401364
[2,]  0.32096272 -0.5721582
```

```
new_assign <- closest_mean(X, means_a)
new_assign
```

```
 [1] 1 2 2 2 1 2 2 2 1 1
```

```
plot(X, col = new_assign + 1, pch = 19, cex = 1.5,
     main = "Observations after one iteration",
     xlab = "x1", ylab = "x2")
points(means_a, col = 2:(K + 1), pch = 8, cex = 2, lwd = 2)
legend("topright", legend = c(paste("Cluster", 1:K), "Mean"),
       col = c(2:(K + 1), "black"), pch = c(19, 19, 8), bty = "n")
```



The matrix gives the two cluster means from the random starting assignment (rows are clusters), the vector gives the updated assignments, and the stars in the plot mark the means used.

b. Write a function that repeats these two steps until the assignments stop changing or a specified maximum number of iterations is reached. Your function should work for a data matrix with any number of variables and a chosen number of clusters K. Start from a nonempty assignment to each cluster; if a cluster becomes empty during iteration, keep its previous mean. Recalculate the cluster means from the final assignments before returning them. Return the final assignments, cluster means, and within-cluster sum of squares,

$$\sum_{k=1}^{K} \sum_{i:\, C(i)=k} \|x_i - m_k\|^2.$$

```
my_kmeans <- function(X, K, init_assign = NULL, max_iter = 100) {
  X <- as.matrix(X)
  n <- nrow(X)

  if (is.null(init_assign)) init_assign <- random_assignment(n, K)
  assign <- init_assign
  means <- cluster_means(X, assign, K)

  iter <- 0
  while (iter < max_iter) {
    iter <- iter + 1
    means <- cluster_means(X, assign, K, prev_means = means)
    new_assign <- closest_mean(X, means)
    if (all(new_assign == assign)) break
    assign <- new_assign
  }

  # Recalculate the means from the final assignments
  means <- cluster_means(X, assign, K, prev_means = means)

  # Within-cluster sum of squares
  wcss <- 0
  for (k in 1:K) {
    if (any(assign == k)) {
      diffs <- sweep(X[assign == k, , drop = FALSE], 2, means[k, ], "-")
      wcss <- wcss + sum(diffs^2)
    }
  }

  list(assignments = assign, means = means, wcss = wcss, iterations = iter)
}
```

c. Apply your function to the same 10 observations from two different random initial assignments. Report the final plots, cluster means, and within-cluster sums of squares. Do the two runs produce the same grouping and objective value? Explain your findings.

```
set.seed(1)
init1 <- random_assignment(n, K)
set.seed(2)
init2 <- random_assignment(n, K)

res1 <- my_kmeans(X, K, init_assign = init1)
res2 <- my_kmeans(X, K, init_assign = init2)

res1$means; res1$wcss
```

```
            [,1]       [,2]
[1,]  0.30797214  0.1617157
[2,] -0.05153971 -2.1409474
```

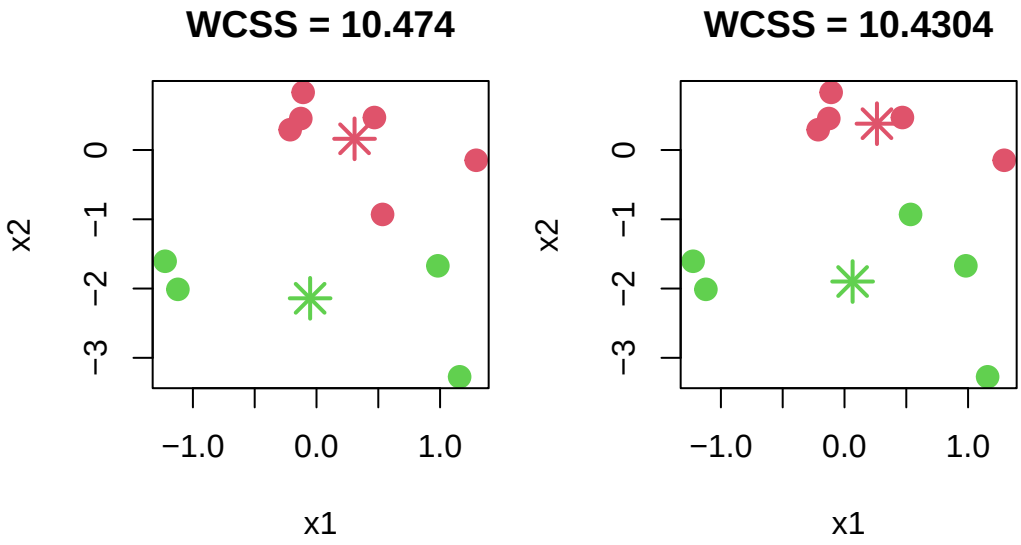
```
[1] 10.474
```

```
res2$means; res2$wcss
```

```
           [,1]       [,2]
[1,] 0.26263713  0.3800466
[2,] 0.06569766 -1.8987456
```

```
[1] 10.43041
```

```
par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
for (r in list(res1, res2)) {
  plot(X, col = r$assignments + 1, pch = 19, cex = 1.5,
       main = paste0("WCSS = ", round(r$wcss, 4)),
       xlab = "x1", ylab = "x2")
  points(r$means, col = 2:(K + 1), pch = 8, cex = 2, lwd = 2)
}
```




```
par(mfrow = c(1, 1))

tab <- table(res1$assignments, res2$assignments)
same_grouping <- all(rowSums(tab > 0) == 1) && all(colSums(tab > 0) == 1)
same_wcss <- isTRUE(all.equal(res1$wcss, res2$wcss))
```

Findings. The two runs produce different groupings, and the within-cluster sums of squares are 10.474 and 10.4304, so the objective values are different.

K-means never increases the within-cluster sum of squares at either step, so it converges, but only to a local minimum, not necessarily the global one. Which local minimum it reaches depends on the starting assignment. If two starts give the same grouping, they reached the same minimum (the labels 1 and 2 are arbitrary). If they give different groupings, they landed in different local minima, and the one with the smaller WCSS is the better solution. This is why K-means is usually run from several random starts and the best result is kept.

Question 2: Image color compression

Pick your favorite color image and use your K-means function from Question 1 to cluster its pixels. For computational efficiency, resize the image to no more than 200×200 pixels and use its three RGB color channels.

```
library(magick)

set.seed(2024)

img_orig <- image_read("/Users/chahakgupta3/Downloads/week-11/my_image.jpg")
info <- image_info(img_orig)
orig_dim <- c(info$height, info$width)

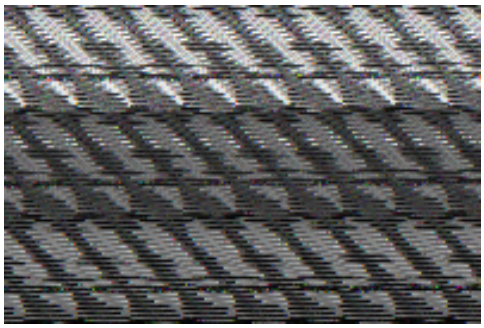
# Resize to at most 200 x 200 (keeps the aspect ratio), 8-bit sRGB
img <- image_resize(img_orig, "200x200")
img <- image_convert(img, colorspace = "sRGB", depth = 8)
```

```
# Array of size height x width x 3 with values in [0, 1]
a <- image_data(img, channels = "rgb")
d <- dim(a)                                    # 3 x width x height
w <- d[2]
h <- d[3]
arr <- aperm(array(as.integer(a), dim = d), c(3, 2, 1)) / 255
```

a. Plot the original image and report its dimensions. Transform it into a data matrix with one observation per pixel and three variables: red, green, and blue. Report the dimensions of this matrix.

```
plot(as.raster(arr))
title("Original image (resized)")
```

Original image (resized)



```
# One row per pixel, columns are red, green, blue
pix <- matrix(arr, ncol = 3)
colnames(pix) <- c("red", "green", "blue")
dim(pix)
```

```
[1] 26800     3
```

The original image is 2000 by 1335 pixels (width by height). After resizing it is 200 by 134 pixels with 3 color channels. The data matrix has 26800 rows (one per pixel) and 3 columns (red, green, blue).

b. For $K = 2, 3, 5, 10$, apply your function to the pixel data. Report the cluster means and within-cluster sum of squares for each K. Reconstruct and plot each compressed image by replacing every pixel with its assigned cluster mean. Use the same RGB scale for all fits.

```
Ks <- c(2, 3, 5, 10)
fits <- list()
for (K in Ks) {
  fits[[as.character(K)]] <- my_kmeans(pix, K, max_iter = 100)
}

# Means and WCSS for each K
for (K in Ks) {
  cat("K =", K, "\n")
  m <- fits[[as.character(K)]]$means
  colnames(m) <- c("red", "green", "blue")
  print(round(m, 3))
  cat("WCSS:", round(fits[[as.character(K)]]$wcss, 2), "\n\n")
}
```

```
K = 2
       red green  blue
[1,] 0.239 0.236 0.238
```

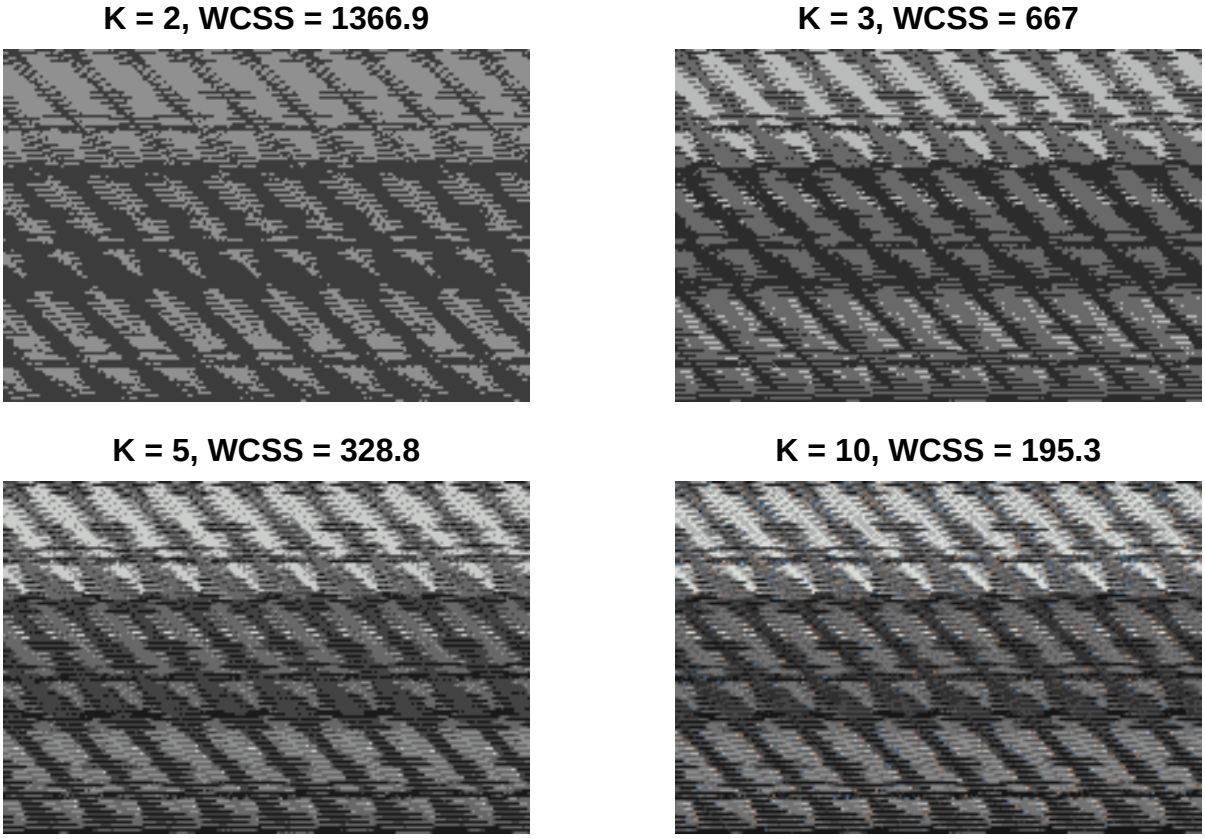
```
[2,] 0.560 0.564 0.561
WCSS: 1366.88

K = 3
       red green  blue
[1,] 0.178 0.174 0.177
[2,] 0.411 0.411 0.411
[3,] 0.727 0.735 0.729
WCSS: 666.96

K = 5
       red green  blue
[1,] 0.408 0.408 0.409
[2,] 0.559 0.562 0.560
[3,] 0.107 0.103 0.106
[4,] 0.266 0.264 0.266
[5,] 0.795 0.805 0.796
WCSS: 328.77

K = 10
        red green  blue
 [1,] 0.365 0.362 0.363
 [2,] 0.552 0.558 0.559
 [3,] 0.608 0.453 0.305
 [4,] 0.281 0.279 0.278
 [5,] 0.848 0.865 0.845
 [6,] 0.702 0.703 0.705
 [7,] 0.192 0.188 0.193
 [8,] 0.446 0.448 0.447
 [9,] 0.084 0.081 0.084
[10,] 0.257 0.411 0.565
WCSS: 195.33
```

```r
par(mfrow = c(2, 2), mar = c(0.5, 0.5, 2, 0.5))
for (K in Ks) {
  f <- fits[[as.character(K)]]
  new_pix <- f$means[f$assignments, ]          # replace each pixel by its cluster mean
  new_arr <- array(new_pix, dim = c(h, w, 3))
  new_arr <- pmin(pmax(new_arr, 0), 1)         # same RGB scale [0, 1] for every fit
  plot(as.raster(new_arr))
  title(paste0("K = ", K, ", WCSS = ", round(f$wcss, 1)))
}
```




```
par(mfrow = c(1, 1))

wcss_tab <- data.frame(
  K = Ks,
  WCSS = sapply(Ks, function(K) fits[[as.character(K)]]$wcss)
)
wcss_tab$pct_reduction_vs_K2 <- round(100 * (1 - wcss_tab$WCSS / wcss_tab$WCSS[1]), 1)
wcss_tab
```

```
   K      WCSS pct_reduction_vs_K2
1  2 1366.8767                0.0
2  3  666.9627               51.2
3  5  328.7696               75.9
4 10  195.3319               85.7
```

c. Which value of K would you choose for this image? Briefly justify your choice using the reconstructed images and the within-cluster sums of squares.

The within-cluster sum of squares drops each time K increases, since more colors can only fit the pixels at least as well. With K = 2 the image only has two colors and loses most of its detail. I would choose K = 5. Going from K = 2 to K = 5 gives a large drop in WCSS and a big visual improvement, while going from K = 5 to K = 10 reduces WCSS by a smaller amount and the images look similar. K = 5 keeps the main color regions using only five colors, which is a good balance between compression and quality.

Question 3: Hierarchical clustering of cars

The file `data/mtcars.csv` contains 11 numerical variables for 32 cars, including fuel economy, horsepower, and weight. The `car` column gives the model names; use it to label observations, not as a clustering variable. You may use existing hierarchical clustering functions for this question.

```
cars <- read.csv("/Users/chahakgupta3/Downloads/week-11/data/mtcars.csv")
vars <- setdiff(names(cars), "car")   # numeric variables only
dim(cars)
```

```
[1] 32 12
```

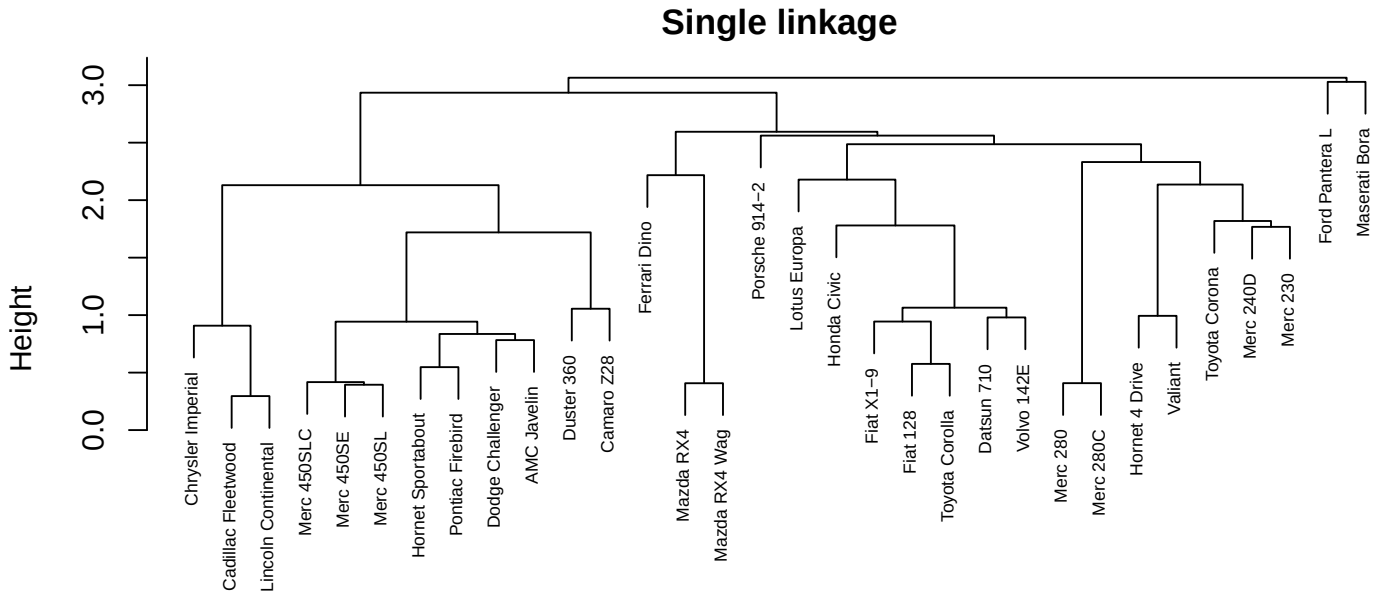
a. Standardize each numerical variable to have mean 0 and standard deviation 1. Using Euclidean distance, perform hierarchical clustering with single, complete, and average linkage. Plot the three dendrograms with the car names and compare the results.

```
X <- scale(cars[, vars])              # mean 0, sd 1 for each variable
rownames(X) <- cars$car

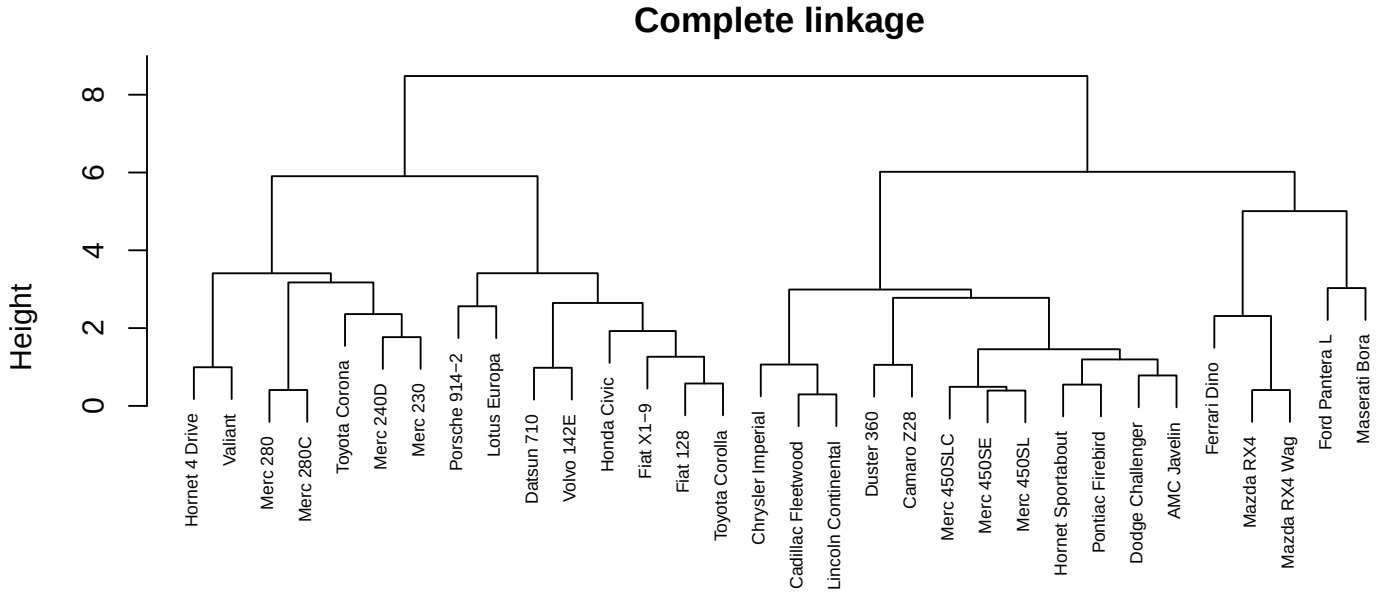
d <- dist(X, method = "euclidean")

hc_single   <- hclust(d, method = "single")
hc_complete <- hclust(d, method = "complete")
hc_average  <- hclust(d, method = "average")
```

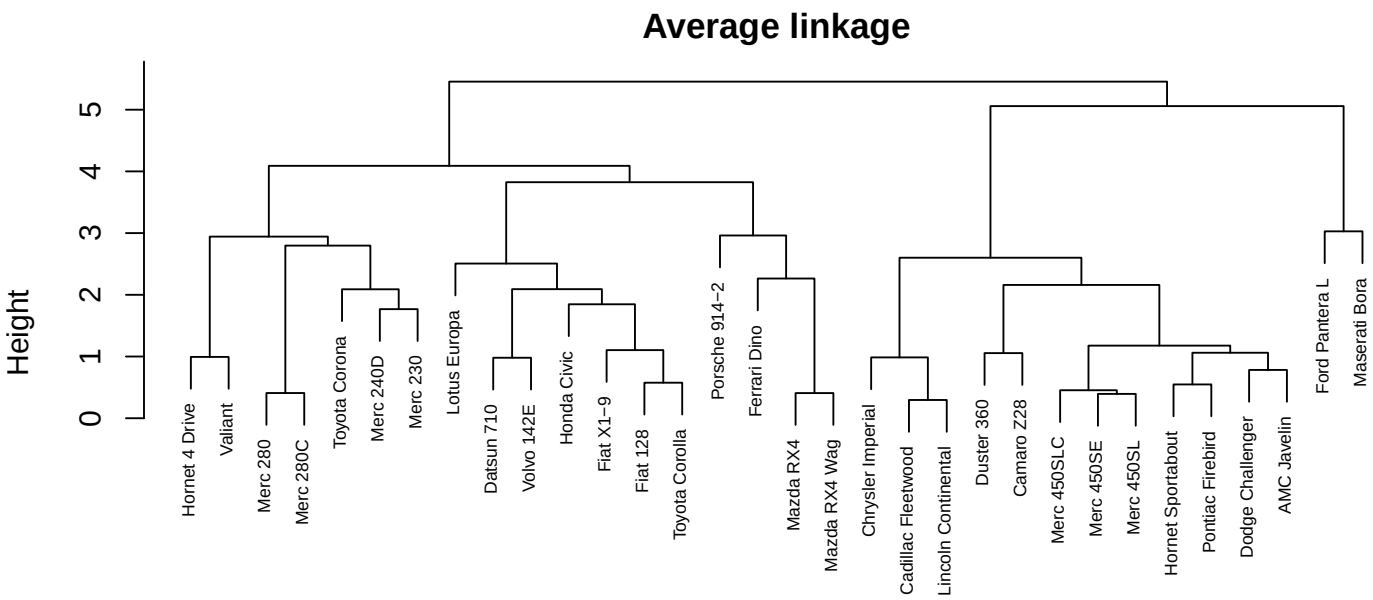
```
par(mar = c(1, 4, 2, 1))
plot(hc_single,   cex = 0.6, main = "Single linkage",   xlab = "", sub = "")
```




```
plot(hc_complete, cex = 0.6, main = "Complete linkage", xlab = "", sub = "")
```



```r
plot(hc_average,  cex = 0.6, main = "Average linkage",  xlab = "", sub = "")
```



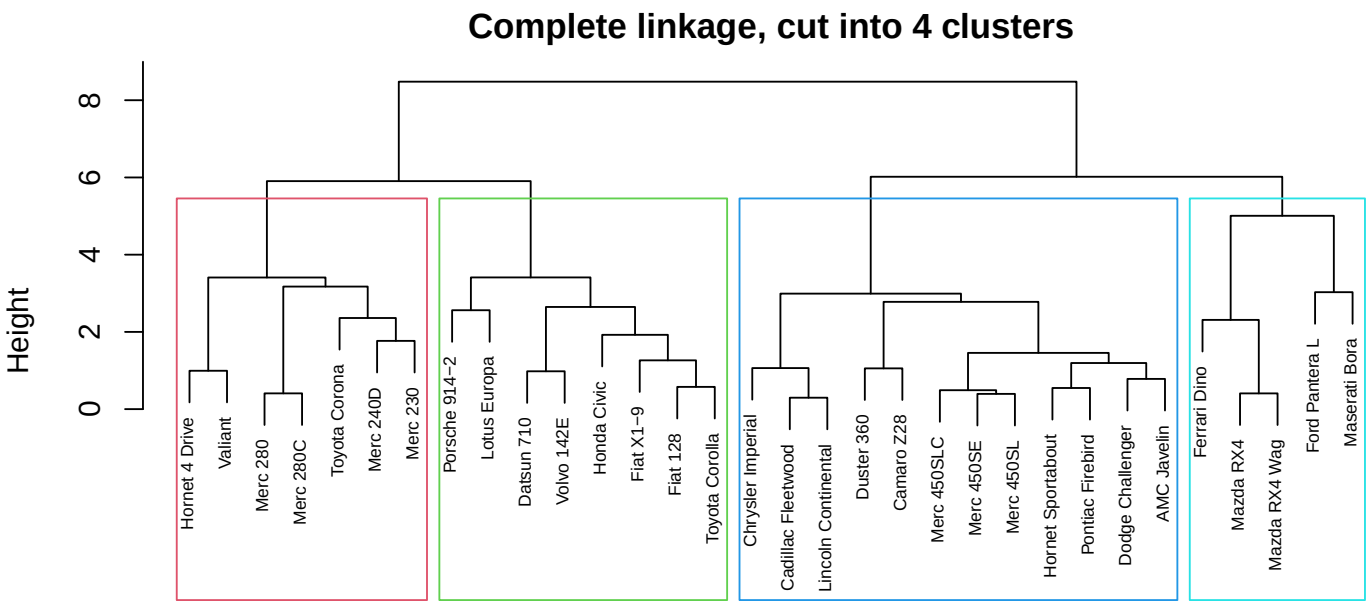
Comparison. Single linkage merges clusters based on their closest pair of points, so it tends to produce long, chained branches where one car at a time joins a growing cluster, which makes clear groups hard to find. Complete linkage uses the farthest pair of points, so it produces more compact, balanced clusters and a clearer split near the top of the tree. Average linkage falls between the two. It is less chained than single linkage but still leaves a few unusual cars (such as the Ford Pantera L and Maserati Bora) joining late. All three methods place similar cars near each other, such as the large V8 cars together and the small economical cars together, but they differ in how the groups are shaped and where the cuts are clearest.

b. Choose one linkage method and a number of clusters that you consider reasonable. Report the cluster memberships and a table of cluster means on the original variable scales. Describe the main characteristics of the groups and explain your choice using the dendrogram and table. Briefly explain why standardizing the variables matters for this analysis.

I choose **complete linkage with $K = 4$ clusters**, because its dendrogram has the most balanced groups and shows large jumps in merge height before the last few merges.

```r
K <- 4
cl <- cutree(hc_complete, k = K)

par(mar = c(1, 4, 2, 1))
plot(hc_complete, cex = 0.6, main = "Complete linkage, cut into 4 clusters",
     xlab = "", sub = "")
rect.hclust(hc_complete, k = K, border = 2:(K + 1))
```




```
# Cluster means on the original scale
means_tab <- aggregate(cars[, vars], by = list(cluster = cl), FUN = mean)
means_tab$n <- as.integer(table(cl))
means_tab[, vars] <- round(means_tab[, vars], 2)
means_tab
```

```
  cluster   mpg  cyl   disp     hp drat   wt  qsec   vs am gear carb  n
1       1 18.50 6.80 223.40 198.80 3.84 3.00 15.62 0.00  1 4.60 5.20  5
2       2 28.08 4.00  93.61  81.88 4.18 2.04 18.45 0.88  1 4.25 1.50  8
3       3 20.74 5.14 175.11 102.14 3.57 3.19 19.97 1.00  0 3.57 2.14  7
4       4 15.05 8.00 357.62 194.17 3.12 4.10 17.14 0.00  0 3.00 3.08 12
```

Description of the groups. Cluster 4 has the lowest average fuel economy (15.05 mpg), with 8 cylinders and a weight of 4.1 (1000 lbs). These are large, heavy cars with big engines. Cluster 2 has the highest fuel economy (28.08 mpg) and is made up of small, light cars with small engines and few cylinders. Cluster 1 has the highest average horsepower (198.8), which separates the high-performance cars from the rest. The remaining cluster falls in between on weight, horsepower, and fuel economy.

Why K = 4. The dendrogram shows clear gaps in merge height before the top few merges, and cutting at four clusters gives groups of reasonable size without isolating single cars. Fewer clusters would combine cars with clearly different engine sizes and weights, and the means table shows that the four groups differ meaningfully on mpg, horsepower, and weight.

Why standardize. The variables are on very different scales: displacement and horsepower take values in the hundreds, while weight, cylinders, and quarter-mile time are in single digits, and `vs` and `am` are just 0 or 1. Euclidean distance adds up squared differences across variables, so without standardizing, displacement and horsepower would dominate the distances. Standardizing gives each variable the same variance, so all of them contribute equally.